Лекция 9: Чистая архитектура (Clean Architecture), Запахи кода (Code Smells) и техники рефакторинга

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Emergence (Кульминация курса: Чистая архитектура и управление эволюцией систем)

Введение и инженерный контекст

Сегодня мы подошли к кульминационной точке нашего семестрового пути. Вспомним методологическую траекторию курса:

Chaos (Процедурное спагетти) → Structure (Классы и инкапсуляция) → Order (SOLID и GoF) → **Emergence** (Чистая архитектура)

В первой половине курса мы изучали правила построения отдельных классов и методов. Во второй — шаблоны их взаимодействия.

Однако в реальных enterprise-системах масштаба **Avtobys** (тысячи валидаторов, мобильные приложения, миллионы транзакций в день) локального качества классов недостаточно. Если в системе нарушены **системные архитектурные границы**, база данных или веб-фреймворк проникают в самое сердце бизнес-логики, делая систему монолитной, нетестируемой и хрупкой.

В этой финальной теоретической лекции мы разберем **Чистую архитектуру (Clean Architecture)** Роберта Мартина, каталог **запахов кода (Code Smells)** Мартина Фаулера и дисциплину безопасного рефакторинга кодовой базы перед защитой финальных проектов.

План лекции (90 минут)

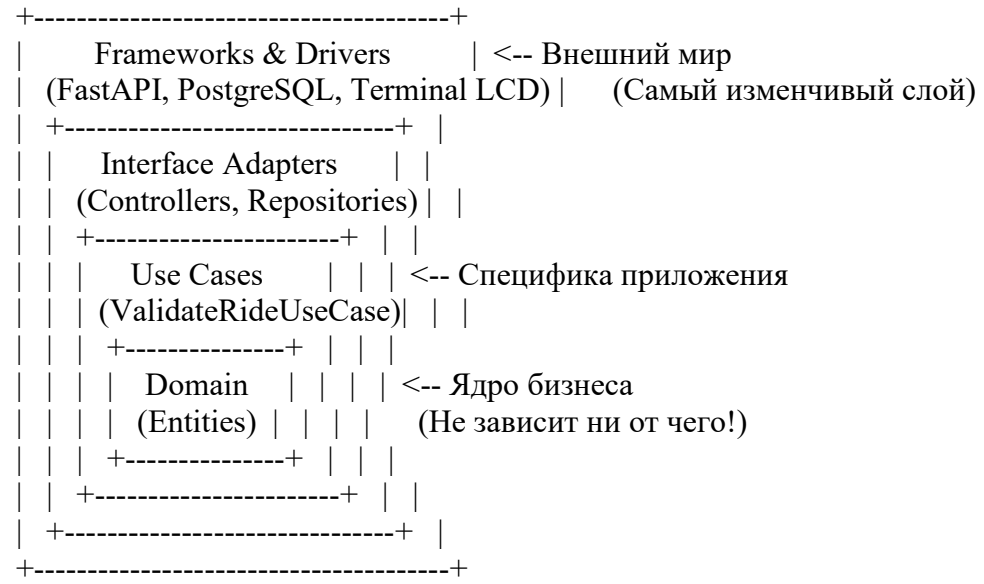
№	Время	Тема	Формат
1	00:00–00:25	Чистая архитектура (Clean Architecture): Правило зависимостей (The Dependency Rule)	Архитектурная теория
2	00:25–00:45	Слои системы: Domain (Entities), Application (Use Cases), Infrastructure, Presentation	Декомпозиция слоев
3	00:45–01:05	Каталог запахов кода (Code Smells)	Анализ симптомов

№	Время	Тема	Формат
		Мартина Фаулера: классификация и диагностика	
4	01:05–01:25	Техники безопасного рефакторинга: тесты как страховочная сетка, пошаговые трансформации	Практика
5	01:25–01:30	Подведение итогов курса и регламент защиты финальных проектов (Неделя 10)	Рефлексия

---

1. Чистая архитектура и Правило зависимостей

Роберт Мартин (Uncle Bob) обобщил идеи Гексагональной архитектуры (Алистер Кокберн) и Луковой архитектуры (Джеффри Палермо), сформулировав концепцию **Clean Architecture**.



Фундаментальное правило зависимостей (The Dependency Rule):

Исходные зависимости в коде должны быть направлены строго ВНУТРЬ — в сторону политик более высокого уровня.

- Внутренние круги ничего не знают о внешних кругах.
- Слой **Domain** не имеет права импортировать ни sqlite3, ни FastAPI, ни requests, ни даже dataclasses_json. Бизнес-правила чисты и независимы от технологий!
- Смена базы данных (например, переход с PostgreSQL на ClickHouse) или смена веб-фреймворка не требует изменения ни одной строчки в бизнес-логике.

---

2. Анатомия слоев чистой архитектуры

1. Доменный слой (domain/):

Содержит корпоративные бизнес-сущности и инварианты: - Сущности: TransitCard, Bus, Route, Ticket. - Объекты-значения (Value Objects): Money, Coordinates, CardId. - Доменные исключения: InsufficientFundsError, InvalidRouteError.

2. Слой сценариев использования (application/use_cases/):

Оркестрирует доменные объекты для выполнения конкретных пользовательских сценариев (Use Cases):

```
# application/use_cases/validate_ride.py
class ValidateRideUseCase:
    """Сценарий валидации проезда пассажира в автобусе."""
    def __init__(
        self,
        card_repo: CardRepositoryPort,
        bus_repo: BusRepositoryPort,
        payment_gate: PaymentGatewayPort,
        notifier: NotificationPort
    ):
        self._cards = card_repo
        self._buses = bus_repo
        self._gate = payment_gate
        self._notifier = notifier

    def execute(self, request: ValidateRideRequest) -> ValidateRideResponse:
        card = self._cards.get_by_id(request.card_id)
        bus = self._buses.get_by_id(request.bus_id)

        # Чистая бизнес-логика выполняется самими объектами домена
        fare = bus.route.calculate_fare(card.category)
        card.charge_fare(fare)

        self._cards.save(card)
        self._notifier.notify_success(card.id, fare)
        return ValidateRideResponse(success=True, fare=fare)
```

3. Слой инфраструктуры (infrastructure/):

Реализует порты (интерфейсы), объявленные в слое application: - SqliteCardRepository (наследует CardRepositoryPort); - KaspiQrPaymentAdapter (наследует PaymentGatewayPort); - TerminalBuzzerDriver (наследует NotificationPort).

---

3. Запахи кода (Code Smells): диагностика архитектурных болезней

Мартин Фаулер в своей классической книге определил запахи кода как **«явные внешние признаки проблем в структуре системы, которые не являются синтаксическими ошибками, но замедляют разработку и ведут к багам»**.

Категория	Запах кода	Симптом и диагностика	Рецепт рефакторинга
Раздувальщики (Bloaters)	*Long Method*	Метод занимает > 30 строк и решает несколько задач	*Extract Method*
	Large Class	Класс содержит десятки полей и нарушает SRP	*Extract Class*
	Primitive Obsession	Использование примитивов (str, float) вместо доменных типов	*Introduce Value Object*
Злоупотребители ООП	*Switch Statements*	Повторяющиеся проверки типов if-elif / match	*Replace Conditional with Polymorphism*
	Refused Bequest	Подкласс переопределяет методы родителя через raise NotImplemented	*Replace Inheritance with Delegation*
Препятствующие изменениям	*Divergent Change*	Один класс меняется по разным причинам от разных людей	Декомпозиция по *SRP*
	Shotgun Surgery	Одно бизнес-изменение требует правок в 15 разных файлах	*Move Method / Move Field*
Связывающие (Couplers)	*Feature Envy*	Метод одного класса постоянно обращается к полям другого класса	*Move Method* в класс с данными

---

4. Дисциплина безопасного рефакторинга

Рефакторинг — это контролируемый процесс изменения внутренней структуры программы без изменения её наблюдаемого внешнего поведения.

Главные законы рефакторинга:

1. **Никогда не рефакторите код без тестов!** Если тестов нет, первым шагом пишутся сквозные тесты (*Characterization Tests / Golden Master*), фиксирующие текущее поведение.
2. **Делайте изменения крошечными шагами**:
   Один мелкий шаг → Запуск тестов (Зеленый) → Коммит в Git

3. **Разделяйте добавление функционала и рефакторинг**: не добавляйте новые фичи во время рефакторинга и не переписывайте архитектуру во время исправления срочного бага.

Подведение итогов курса и подготовка к Неделе 10

На следующей 10-й неделе состоится **Публичная защита семестровых проектов**: - Каждый студент демонстрирует работающую кодовую базу системы Avtobys, выстроенную по Clean Architecture; - Защита сопровождается UML-диаграммами классов и последовательностей; - Проверяется 100% прохождение тестов pytest и отсутствие запахов кода.

Литература к лекции

1. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021. (Все части).
2. **Фаулер М.** Рефакторинг: улучшение проекта существующего кода. – 2-е изд. – М.: Диалектика, 2020. (Главы 1–3: Принципы рефакторинга и каталог Code Smells).
3. **Cockburn A.** Hexagonal Architecture (Ports and Adapters): https://alistair.cockburn.us/hexagonal-architecture/